Ternary Trivia Project Report

An analysis of developing a Jeopardy-like program

Prepared by: Hunter Hayden, James Sturdivant, Ryan Edimo, Jack Harris

Date: April 12, 2026

Table of Contents

GitHub Links 3

Project Report 3

1. Abstract 3

2. Introduction 4

    2.1 Project Idea Decision Making 4

3. Rules of the game 5

4. GitHub Usage 5

5. Visuals 6

6. Question and Game Editor 6

7. Game Behavior 6

    7.1 Version 1-1.3 6

    7.2 Version 1.4 7

    7.3 Version 1.5 9

8. File Handling 10

    8.1 Version 1 11

    8.2 Version 2 13

    8.3 Version 3 14

9. Conclusion 15

10. Sources 16

11. Contributions 16

GitHub Links

Ternary Trivia Repo

Project Report

1. Abstract

For this project, we decided to create a Jeopardy-style trivia game where there would be a series of five categories and five questions for each category. They would be worth 100-500 points, increasing by 100 each step, which also increases the difficulty of the question. The precise behavior and rules set will be elaborated on further later in the report. This version of our game has an interface that is displayed in the console/terminal, similar to how it is displayed in the game show, with text wrapping, as we aren't sure of the definite lengths of questions and answers. The user has to type out the item they want to select to interact with the program, which has error checking, so it continuously prompts the user until a valid input is received or the program is terminated. We chose this idea because we wanted to create a fun game that was semi-simple but also allowed for more complexity if we were able to complete each individual piece.

2. Introduction

This project came from multiple ideas coming together to morph into this after an hour of pinging ideas back and forth off of each other until we came up with a solid idea for the project. Ternary Trivia is the name of our group's game, and it behaves similarly to Jeoparty, where there are multiple teams who compete against each other to see who has the most Trivia knowledge out of five categories, all of which have more difficult questions per their point values. The goal of the game is to have the most points by the end of the game, declaring you the winner. The end of the game is when there are no more questions for a person to answer.

2.1 Project Idea Decision Making

Initially, we had a handful of ideas, which included a text-based adventure, a weather system that would email you an idea of what today's weather would look like, a calendar where you could add events, a GPA calculator, and something that would help you select what classes would be best to take next in your degree pathway.

We decided on none of these ideas as all of us were not fully behind them for some reason or another, so we decided to take parts of each and figure out what we liked from them to refine and narrow down our idea of what we wanted to do. After conversing, we figured out that a quiz game would allow for the implementation of multiple characteristics that we liked while also not being overtly complex and allowing for future game improvements and iterations if we could manage to get each individual component done first.

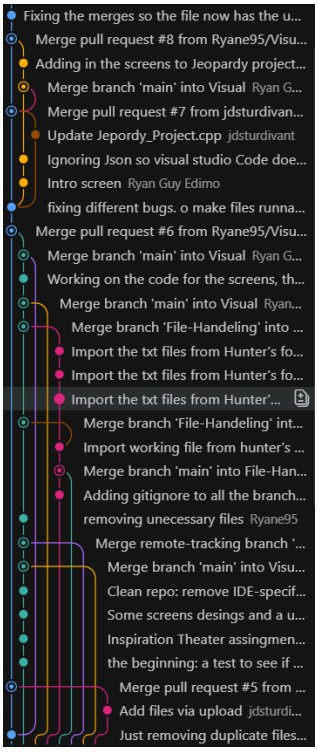
We broke the game down into four parts so that each member of our group could focus on a specific task, and we could compile it all together when each piece is a manageable chunk and can be easily put together. We broke it down into Visuals, Question and Game Editor, Game Behavior, and File Handling. We will explain the behaviors of each segment in the following sections.

3. Rules of the game

Teams choose their own names and begin with zero points. The host may use the provided questions and answers or create new ones, including three incorrect options for each question. Teams take turns selecting a row and column on the board, after which they are presented with a multiple-choice question. Turn order is randomized at the start of the game. Each team answers during its turn, and play continues until the host decides to end the game. The team with the highest score at the end is declared the winner.

4. GitHub Usage

Over the course of this project, we implemented Git Hub as it was a simple addition to make file sharing significantly better than any other option out there. We implemented several features, including pushing, pulling, creating branches, forming pull requests, and repository forking. This helped significantly, as it made it much easier for our group to work off the same files and complete the final implementation of adding each component together.



5. Visuals

The idea for the visuals was to have a six-row and five-column layout so that one of the rows has the titles of the categories. This evolved into having the user select one of the values in that category that had not been selected previously from a five-by-five row of cells.

As we weren’t sure how long each question or answer would be, we decided to have the functionality of line wrapping implemented. If the line is too long to fit inside the length of the box, then the program detects that and starts printing it out on another line, so that we effectively use the terminal space.

6. Editor

The idea behind the editor is to give the players the opportunity to customize their experience and add their own categories and questions. This will allow players the capability to do more than the small scope of our project. Currently the editor only allows the addition of new questions and doesn't allow for the deletion of old ones. The editor keeps track of the newly created category by using the subjects.txt text file. Where it writes the name of the new file onto it so it can be accessed without having to hardcode the names of new files. When adding questions to a category the editor will ask for all 6 pieces of information required for a new question and write it to the file.

7. Game Behavior

This program asks the user to pick if one would like to use preloaded questions or chosen questions, after that users will choose between four answers, the host can decide to show score, have players guess, change points, and exit the program. As a team, it was decided that the best course of action would be to have a turn counter to decide who guesses at the given time, once the board is clear the host can either re-randomize the files or create more questions and answers. Initially, the idea was a simple 2d array with the row and col holding the answers and a second 2d array to hold questions.

7.1 Version 1-1.3

In Version 1, the game logic was built around several independent 2D arrays, one for questions, one for answers, one for point values, and one for flags. The main function relied on while-loops to handle user input and move through the game. Versions 1.2–1.3 marked the transition to a more organized structure-based design. Instead of juggling multiple arrays, all question data was stored in a single struct, arranged in a 2D array representing the board. These versions also introduced a switch statement for user prompts and added placeholder functions that would later handle validation and gameplay operations. Below are some examples of the previous code for the initial structure of the code.

```
board jepordy[5][5];
//r for request
string r;
int i,j, choice;
bool creative = false;


cout << "Would you like to populate your own questions, or get a random category and question (y/n)? ";
cin >> r;
while (r != "y" || r != "Y" || r != "n" || r != "N")
{
    if (r == "y" || r == "Y")
        creative = true;
    else if (r == "n" || r == "N")
        creative = false;
    else
        {
            cout << "Pleases enter y/Y for yes or n/N for no. ";
            cin >> r;
        }
}
//Found cin >> ws; on https://stackoverflow.com/questions/58705197/cin-ws-vs-cin.ignorenumeric-limitsstreamsizemax-n
//clears leftover characters until white space good for getline.
cin >> ws;
while (creative == true)
{
    for (i = 0; 5 >= i; i++ )
    {
        for (j = 0; 5 >= j; j++)
        {
            cout << "Please provide your question.";
            getline(cin, jepordy[i][j].question);
            cout << "What is the answer to your question?" ;
            getline(cin, jepordy[i][j].answer);
        }
    }
}
```

```
do
{
    displayMenu();
    choice = getChoice();

    switch (choice)
    {
    case 1: answerQuestion(teams, jepordy);
        break;

    case 2: showScore(teams);
        break;

    case 3: restartGame(teams, jepordy);
        break;

    case 4: changePoints(teams);
        break;

    default: // Do nothing
        break;
    }

} while (choice != 5);

return 0;
}
```

7.2 Version 1.4

In Version 1.4, nine functions were added: four for host interactions and five for background logic. Teams were moved into their own structure with associated point values. Minor fixes included converting some while-loops to if-else logic, improving validation by using and instead of or statements, updating main to produce three fake answers, and replacing ws with ignore for cleaner input processing.

```
int main()
{
    //Make sure to give a input validate function.
    team team1, team2, team3;
    team teams[3];

    //Add a file and import it into struct

    //Trying out structures with array
    board jepordy[5][5];
    //r for request
    string request;
    int choice;
    bool creative = false;

        for (int i = 0; i < 3; i++)
        {
            cout << "Team " << i + 1 << " is called? ";
            getline(cin,teams[i].name);
        }
        shuffleTeams(teams);
        loadPoints(jepordy);

    cout << "Would you like to populate your own questions, or get a random category and question (y/n)? ";
    while ((cin >> request) && request != "y" && request != "Y" && request != "n" && request != "N")
    {
        cout << "Pleases enter y/Y for yes or n/N for no. ";
    }
        if (request == "y" || request == "Y")
            creative = true;
        else if (request == "n" || request == "N")
            creative = false;

        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        if (creative == true)
        {
            populateOwnQuestions(jepordy);
        }
```

This updated version also had a way for the host to prepare their own questions, answers, and three fake answers. Prompting the user twenty-five times for a five-by-five board. In which the answers and fake answers would be shuffled later on in the code. The image below shows the function to create questions, answers, and fake answers to fill the board. The bool creative is used to call the function, using an if flag is true statement, and later in the function the use of another function called restartGame sends the flag creative for the host to pick once again. This function was replaced by the editor.

```
void populateOwnQuestions(board jepordy[5][5])
{
    bool creative;
        for (int i = 0; i < 5; i++)
        {
            for (int j = 0; j < 5; j++)
            {
                cout << "Please provide your question: ";
                getline(cin, jepordy[i][j].question);
                cout << "What is the answer to your question: ";
                getline(cin, jepordy[i][j].answer);
                cout << "Please enter three fake answers: " << endl;
                getline(cin, jepordy[i][j].fake1);
                getline(cin, jepordy[i][j].fake2);
                getline(cin, jepordy[i][j].fake3);

            }
        }
        creative = false;
```

The host is also prompted for options to show scoring, restart the game, answer questions, and change point values if the host believes the players were falsely given points or had their points removed by answers deemed not correct by the host. Along with the ability for the program to exit by the host's decision.

7.3 Version 1.5-1.6

Version 1.6 is the polished version of the code, adding three new functions, including a turn order system, a point value loader, and a team randomization for which team starts first. Version 1.6 mainly focuses on bug fixes and final touches. The code introduced the function loadPoints, shuffleTeams, and order. Below are the functions to load the points inside the 2d array jeopardy, shuffle the teams, and then put them inside a turn order.

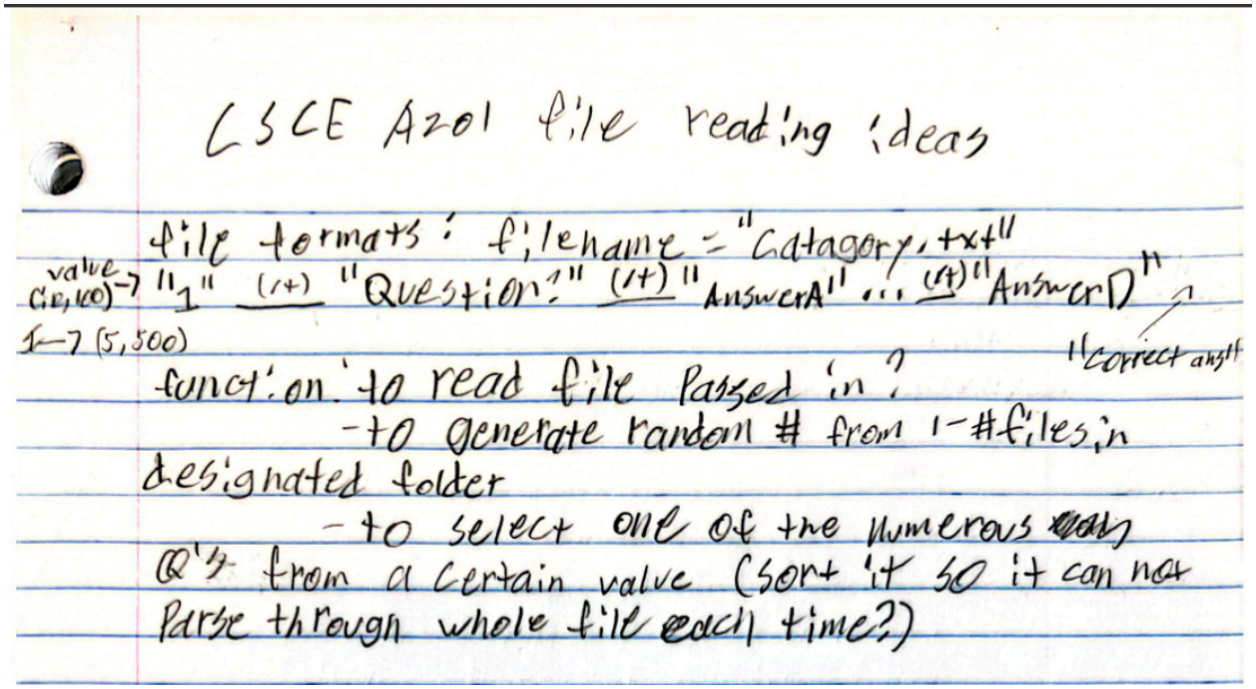
```
void loadPoints(board jepordy[5][5])
{
    for (int row = 0; row < 5; row++)
        for (int col = 0; col < 5; col++)
    {
        jepordy[row][col].points = (row + 1) * 100;
    }
}

void shuffleTeams(team teams[])
{
    srand(time(0));
    for (int i = 0; i < 3; i++)
    {
        int j = rand() % 3;
        team temp = teams[i];
        teams[i] = teams[j];
        teams[j] = temp;
    }
}
int order()
{
    static int current = 0;
    int index = current;
    current = (current + 1) % 3;
    return index;
}
```

8. File Handling

The initial ideas for the file handling case were to try to find a way to make a file window pop up when running the program and allow the user to search through their file directories and select which .txt files that they would like to use for the game. We

wanted to have tab-separated values for the questions, answers, and point values. All while having more than five questions in each section and randomly selecting one of those questions, so each play-through would allow the player to experience a different series of questions. The figure below is a photo of initial file handling ideas, as well as areas for future improvements we would like to attempt if we get basic functionality working.



Above is a general idea of what functionality the function reading behavior should be followed in the program to allow the ability to effectively handle the game.

8.1 Version 1

For the first iteration of file handling, it was all in a singular file to test basic functionality. Vectors, strings, and a random number to choose one of the questions in the .txt files.

Below is a photo of the assumed initial format, where the spaces represent tabs.

```
/*****************************************
 * file format should be the following
 * fiel name : "category.txt" i.e. "Math.txt" or "Sci-fi history.txt"
 * "value (1-5) indicating value/points/difficulty" "Question" "answer1" ... "answer 4" "correct answer"
 ***************************************** */
```

Below is the main file reading functionality. The readFile function is responsible for reading in and processing the quiz question file. It takes a file name as the parameter and uses an input file stream object to make sure we don't change any of the data in the file. If the file isn’t able to be opened, then the function outputs an error message and returns from the function. After the file is properly opened, the first line is read and output to the console. Next, a two-dimensional vector of strings called questions is created to store the data from each question, where indices 1-5 are the difficulty of each question.

Then, readFile reads each remaining line of the file, where each line represents a question entry containing the value, question text, four answer choices, and the correct answer. Then the line is split into individual components using the splitLine function. If the line contains the correct number of elements, then the first value is converted to an integer and used as the index to store the question data. Lastly, the function iterates through the questions to output each of the lines of the questions, the value/weight of the question, the question itself, the answer options, and the correct answer, and proceeds to close the file.

```
void readFile(string filename) // read in the file
{
    ifstream file(filename);
    if (!file)
    {
        cout << "Error opening file: " << filename << endl;
        return;
    }

    string category;
    getline(file, category); // first line is category  i.e.
    /*
    "Sci-fi history.txt" THIS PART!
    "value (1-5) indicating value/points/difficulty" "Question" "answer1" ... "answer 4" "correct answer"
    */

    cout << "\nCategory: " << category << endl;

    vector<vector<string>> questions(6); // index 1-5

    string line;
    while (getline(file, line))
    {
        vector<string> parts = splitLine(line);

        if (parts.size() >= 7)
        {
            int value = stoi(parts[0]); // value == 1-5  stoi is str2int https://cplusplus.com/reference/string/stoi/
            questions[value] = parts;
        }
    }

    for (int i = 1; i <= 5; i++)
    {
        if (!questions[i].empty())
        {
            cout << "\nValue: " << questions[i][0] << endl;
            cout << "Question: " << questions[i][1] << endl;

            cout << "Choices: ";
            for (int j = 2; j <= 5; j++)
            {
                cout << questions[i][j] << " ";
            }

            cout << "\nCorrect Answer: " << questions[i][6] << endl;
        }
    }

    file.close();
}
```

8.2 Version 2

Version 2 added the ability for the program to continuously prompt the user for a valid answer 1-4 for the question, allowing the user's input to be recognized and the program to respond with whether that answer was correct or not. In addition to the prior, the second version added a pause for the game, waiting for the user to hit the enter key to continue playing the game and review their previous answers. Alongside that, it

added several basic .txt files with simple questions such as “what’s 1+1?” in “Math.txt” alongside others such as “Science.txt”, “Movies.txt”, among others.

```
int getInput(int min, int max)
{
    int choice;

    while (true)
    {
        cout << "Enter a number (" << min << "-" << max << "): ";
        cin >> choice;

        if (cin.fail()) // checks if user typed something non-numeric
        {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "Invalid input. Please enter a number.\n";
        }
        else if (choice < min || choice > max)
        {
            cout << "Please enter a number between " << min << " and " << max << ".\n";
        }
        else
        {
            return choice;
        }
    }
}
```

8.3 Version 3

Version three of the file handling code changed the code to run out of a header and main .cpp file so that whenever all the pieces were connected together, we would be able to copy those files and implement only the pieces we needed, making it modular and easy to update. Additionally, a Question struct was added, which stored the value, question, question answers, and the correct answer in one item. The main functionality wasn’t largely changed to accommodate the change.

```
struct Question
{
    int value;
    string text;
    vector<string> answers;
    int correctIndex;
};

int getRandom(int max);
vector<string> splitLine(string line);
void readFile(string filename);
int getInput(int min, int max);
void pauseGame();

#endif
```

9. Conclusion

In conclusion, we were able to implement the main game functionality, the file reading, and the Visuals for the program. We see this as a success, even though there was more we wanted to accomplish with this. We are proud of the result and wish to do more with it in the future. Additionally, for future ideas, we would want to add more .txt files, music, and possibly even add seasonal files using time to tell what season it is. We could get the month and have a ‘Halloween’ or ‘Christmas’ seasonal file.

More on future ideas, another idea we had but didn’t have the chance to implement was to buzz in in some fashion. Another idea was to make the program in a Graphical User Interface instead of the inline format that we currently have.

Overall, we achieved our initial goal of creating a Jeopardy-like program that would read in text files or get the user to create a .txt file in which the user creates a series of questions, answers, the value of the question, and correct answers.

10. Sources

C++ Website: https://cplusplus.com/, used for general syntax and included header files

- strtok(), vectors, etc.

Structures in C++ Structures in C++ - GeeksforGeeks for general use, and learning purposes.

C++ c++ - cin >> ws vs cin.ignore(numeric_limits<streamsize>::max(), '\n')? - Stack Overflow, ws was originally used but later ignore.

11. Contributions

Hunter:

- Programming: File input and reading (with subsections), GitHub Usage (Created a repository, pushed & pulled files, created pull requests)
- Paper: Abstract, introduction, file handling (with subsections), sources, conclusion, formatting, GitHub Usage
- Presentation: Themes, file handling, introduction, GitHub usage, Conclusion, Sources

James:

- Game design: Created the main code.
- Paper: Game Behavior, Rules of the game, sources
- GitHub worked on main, created branches for jeopardy.cpp
- Presentation: Game Rules, Question and Game Editor

Jack:

- Programming: Question/Game Editor
- Paper: Editor
- Presentation: Editor

Ryan:

- Programming: Visuals, GitHub help, created the repo, handled all pull requests, created the .README)
- Paper: Visuals
- Presentation: Visuals